COLLEGE OF ENGINEERING VIZIANAGARAM

Department of Information Technology

M.TECH / QUANTUM COMPUTING

LECTURE NOTES

UNIT IV: Quantum Programming & Simulation Platforms

Course Structure: 300 Minutes / 5 Hours

Unit IV — Syllabus Coverage

This unit moves from quantum-computing theory to practical programming and execution. It introduces major quantum SDKs, Qiskit programming, classical simulators and cloud quantum processors, hardware-aware execution, noise and decoherence, noise mitigation, and introductory quantum error correction. The unit concludes with an integrated workflow from writing a circuit to interpreting experimental results.

Topics Covered

- Introduction: From Theory to Running Code
- The Quantum SDK Landscape: Qiskit, Cirq, Q#, PennyLane, Braket and OpenQASM
- Hands-On Qiskit Fundamentals
- Simulators vs Real Hardware (Cloud QPUs)
- Noise and Decoherence
- Noise Mitigation and Circuit Compilation
- Quantum Error Correction Basics
- Integrated Lab Walkthrough
- Unit Recap & Self-Assessment

Unit Learning Outcomes

1. Explain how quantum SDKs connect quantum algorithms, circuits, simulators and real hardware.
2. Describe the roles of Qiskit, Cirq, Q#, PennyLane, Braket and OpenQASM.
3. Create, execute and inspect simple quantum circuits using Qiskit.
4. Distinguish statevector/unitary/QASM-style simulation from execution on a cloud QPU.
5. Explain qubit connectivity, native gates, transpilation, T1, T2 and common hardware errors.
6. Describe common noise channels and compare noiseless and noisy circuit results.
7. Explain basic noise-mitigation techniques and circuit-compilation strategies.
8. Explain the motivation and basic workflow of quantum error correction using repetition codes and the surface-code concept.
9. Describe an end-to-end quantum workflow: write → transpile → simulate → run on hardware → mitigate → interpret.

Part A: Quantum Programming & Simulation Platforms

A.1 Introduction: From Theory to Running Code

Quantum computing becomes practical when mathematical descriptions of states, gates and algorithms are converted into executable circuits. A quantum software workflow normally connects four layers: an application or algorithm, a quantum SDK, a simulator or hardware backend, and a classical environment used to prepare inputs and analyze results.

A.1.1 From Algorithm to Hardware

- Algorithm: defines the computational task, such as search, phase estimation or optimization.
- Circuit: represents the algorithm as a sequence of quantum gates and measurements.
- SDK: provides programming abstractions for constructing, compiling and executing circuits.
- Simulator: executes a mathematical model of the quantum system on a classical computer.
- Real QPU: executes the circuit on physical qubits and returns experimental measurement results.
- Post-processing: classical software interprets counts, probabilities and application-level results.

A.1.2 NISQ-Era Workflow

Near-term quantum devices are noisy and have limited connectivity and coherence. Therefore, a realistic workflow must consider circuit depth, hardware connectivity, native gates, calibration data, noise and measurement error rather than treating an ideal circuit as automatically executable on hardware.

A.2 The Quantum SDK Landscape

Platform / Tool	Main Role	Typical Use
Qiskit	Quantum SDK and ecosystem	Circuit construction, simulation, transpilation and IBM Quantum execution
Cirq	Quantum circuit framework	Circuit design and hardware-oriented experiments
Q#	Quantum programming language	Quantum algorithm development in Microsoft's ecosystem
PennyLane	Quantum machine-learning / differentiable framework	Hybrid quantum-classical models and variational algorithms
Amazon Braket	Cloud quantum service	Access to simulators and quantum hardware from a cloud environment
OpenQASM	Quantum assembly representation	Representing quantum circuits for interoperability and execution

A.2.1 Qiskit

Qiskit provides programming abstractions for constructing quantum circuits, applying gates and measurements, transpiling circuits for target backends, simulating circuits, and accessing IBM Quantum resources.

A.2.2 Cirq

Cirq is a Python-based framework focused on constructing and manipulating quantum circuits, particularly with hardware-oriented control over gates, qubits and circuit structure.

A.2.3 Q#

Q# is a quantum programming language designed for expressing quantum algorithms and integrating quantum operations with classical programs.

A.2.4 PennyLane and Braket

PennyLane emphasizes hybrid quantum-classical computation, automatic differentiation and quantum machine learning. Amazon Braket provides cloud access to quantum simulators and hardware from multiple providers.

A.2.5 OpenQASM

OpenQASM is a quantum assembly representation used to describe quantum circuits in a machine-readable form. It can serve as an intermediate representation between high-level programming and lower-level execution systems.

A.3 Choosing an SDK

Criterion	Question to Ask
Hardware access	Which real quantum processors or simulators can be accessed?
Programming model	Is Python, Q#, or another language preferred?
Compilation	What transpilation and optimization tools are available?
Hybrid workflows	Does the platform support classical optimization and differentiable circuits?
Learning resources	Are documentation, tutorials and examples available?
Interoperability	Can circuits be represented or exported in standard forms?

Part B: Hands-On Qiskit Fundamentals

B.1 Environment and Circuit Model

A Qiskit program typically creates quantum and classical registers, builds a QuantumCircuit, applies gates, adds measurements when required, and sends the circuit to an appropriate simulator or hardware backend.

B.1.1 Basic Circuit Structure

10. Create quantum and classical registers.
11. Create a QuantumCircuit.
12. Apply gates such as H, X, Z or CX.
13. Add measurements when a classical result is required.
14. Choose a simulator or hardware backend.
15. Execute the circuit and inspect counts or other results.

B.1.2 Simple Bell-State Example

Start with $|00\rangle$. Apply a Hadamard gate to the first qubit and then a CNOT with the first qubit as control and the second as target.

State evolution: $|00\rangle \rightarrow (|00\rangle + |10\rangle)/\sqrt{2} \rightarrow (|00\rangle + |11\rangle)/\sqrt{2}$.

When measured in the computational basis, the ideal result is approximately 50% 00 and 50% 11 over many shots.

B.1.3 What to Inspect

- Circuit diagram: shows the sequence and placement of gates.

- Counts: records how often each classical bit string occurs.
- Statevector: represents amplitudes for an ideal pure state when supported by the selected simulator.
- Unitary: represents the overall matrix operation of a circuit when applicable.
- Histogram: visualizes measured outcomes.

B.2 Guided Programming Exercise

Task: construct a two-qubit Bell circuit, simulate it for 1024 shots, plot the measurement histogram, and explain why only two dominant outcomes are expected in the ideal case.

Expected observation: the outcomes 00 and 11 should dominate, while small deviations can appear in practical or noisy execution.

Part C: Simulators vs Real Hardware

C.1 Classical Quantum Simulation

A classical computer can simulate a quantum circuit by storing and manipulating a mathematical representation of the quantum state. For n qubits, a general statevector contains 2^n complex amplitudes. Consequently, memory and computation requirements grow rapidly with qubit count.

C.1.1 Example of Statevector Size

Qubits (n)	Amplitudes (2^n)
2	4
10	1,024
20	1,048,576
30	1,073,741,824

This exponential state representation is a major reason why large, general-purpose quantum systems are difficult to simulate exactly on classical machines.

C.2 Real QPUs Over the Cloud

A cloud quantum processor accepts submitted jobs through a service interface. A typical job may wait in a queue, be compiled or transpiled for the selected device, execute for a requested number of shots, and return measurement data.

C.2.1 Hardware Properties

- Qubit count: number of physical qubits available.
- Connectivity map: identifies which qubits can directly interact.
- Native gate set: operations physically supported or efficiently implemented.
- Gate error rates: estimates of imperfections in gate operations.
- Readout error: probability of incorrectly recording a measured state.
- T1: characteristic energy-relaxation timescale.
- T2: characteristic dephasing/coherence timescale.

C.3 Transpilation

A circuit written using abstract gates may not match a device's native gate set or connectivity. Transpilation transforms the circuit into hardware-compatible operations while attempting to preserve its logical function and reduce execution cost.

Typical transformations include gate decomposition, qubit mapping, routing through available couplers, cancellation of redundant gates, and optimization of circuit depth.

C.4 Simulator vs Hardware

Aspect	Ideal Simulator	Real QPU
State evolution	Mathematical / idealized	Physical quantum evolution
Noise	Absent unless a noise model is added	Naturally present
Connectivity	Can be abstract	Fixed by hardware
Gate set	Flexible model	Hardware-dependent
Results	Can match exact probabilities	Statistical experimental results
Execution queue	Usually local	May require cloud queue

Part D: Noise and Decoherence

D.1 Why Qubits Are Fragile

Physical qubits interact with their environment. Unwanted interactions can change amplitudes, destroy phase relationships, introduce gate errors, or cause incorrect measurement outcomes. These effects are collectively described through noise and decoherence.

D.2 T1 and T2

Quantity	Meaning	Effect
T1	Energy-relaxation timescale	Excited-state population tends to decay toward the ground state
T2	Dephasing/coherence timescale	Relative phase information becomes less reliable

Shorter coherence times constrain how long useful quantum information can remain available without significant degradation.

D.3 Common Noise Channels

Noise Channel	Basic Effect
Bit flip	Transforms $\|0\rangle \leftrightarrow \|1\rangle$, associated with an X-type error
Phase flip	Changes relative phase, associated with a Z-type error
Depolarizing	Randomizes the state toward a mixed state
Amplitude damping	Models energy relaxation from excited to lower-energy states

D.4 Sources of Practical Error

- Gate errors caused by imperfect control operations.
- Readout errors caused by imperfect measurement.
- Crosstalk caused by unwanted interactions between nearby qubits or control channels.
- Decoherence caused by interaction with the environment.
- Connectivity and routing overhead introduced during hardware-aware compilation.

D.5 Density Matrices — Introduction

Pure states can be represented by statevectors. A density matrix provides a more general description that can represent both pure and mixed states and is particularly useful when describing noisy quantum systems.

For a pure state $|\psi\rangle$, the density matrix is $\rho = |\psi\rangle\langle\psi|$. Noise channels can transform ρ into a mixed state, allowing probabilistic uncertainty and environmental effects to be represented mathematically.

D.6 Noiseless vs Noisy Simulation

A useful experiment is to execute the same circuit first with an ideal simulator and then with a simulator containing a noise model. Comparing the resulting histograms shows how gate, readout and decoherence effects change the observed distribution.

Part E: Noise Mitigation and Circuit Compilation

E.1 Why Mitigation Is Needed

Quantum error correction aims to protect logical information by encoding it redundantly, while noise mitigation attempts to reduce the impact of errors on measured results without fully encoding every logical qubit into a large error-correcting code. Both are important approaches for noisy quantum computing.

E.2 Measurement Error Mitigation

Measurement calibration can estimate how often each physical state is reported incorrectly. A calibration matrix can then be used to reduce systematic readout bias in the observed results.

E.3 Zero-Noise Extrapolation

Zero-noise extrapolation evaluates a circuit under different effective noise levels and uses the resulting measurements to estimate what the result would be at a lower or ideally zero noise level.

E.4 Probabilistic Error Cancellation

Probabilistic error cancellation models noisy operations and combines experimentally sampled results so that error contributions can be statistically reduced. The method can introduce sampling overhead.

E.5 Dynamical Decoupling

Dynamical decoupling applies carefully chosen control operations during idle periods to reduce the effect of certain environmental interactions and preserve quantum coherence.

E.6 Circuit Compilation Strategies

- Reduce circuit depth where possible.
- Cancel redundant or adjacent inverse operations.
- Map logical qubits to physically connected qubits.
- Reduce the number of inserted routing operations.
- Prefer suitable native gates supported by the target hardware.
- Use compilation and optimization settings appropriate to the device.

Part F: Quantum Error Correction Basics

F.1 Why Quantum Error Correction Is Different

Classical error correction can copy and compare bits. Quantum information cannot simply be copied because of the no-cloning theorem, and quantum errors can affect continuous amplitudes and phases. Quantum error correction therefore encodes information across multiple physical qubits and extracts error syndromes without directly measuring the encoded logical state.

F.2 Pauli Error Basis

For introductory quantum error correction, arbitrary single-qubit errors can be decomposed into combinations of the Pauli operators I, X, Y and Z. Therefore, learning how to detect and correct X- and Z-type errors provides the basic foundation for correcting general single-qubit errors.

F.3 Three-Qubit Bit-Flip Repetition Code

The logical states are encoded as $|0L\rangle = |000\rangle$ and $|1L\rangle = |111\rangle$. If one physical qubit flips, the three physical bits no longer agree. Syndrome measurements can identify which qubit is inconsistent, after which the corresponding correction can be applied.

Example

Suppose $|000\rangle$ is encoded and the second qubit experiences an X error: $|000\rangle \rightarrow |010\rangle$. Comparing parity relationships identifies the second qubit as the likely error location, allowing an X correction to recover $|000\rangle$.

F.4 Three-Qubit Phase-Flip Code

A phase-flip error is a Z-type error. A phase-flip repetition code can be constructed by changing to an appropriate basis so that phase errors can be treated analogously to bit-flip errors.

F.5 Nine-Qubit Shor Code

The nine-qubit Shor code combines repetition-code ideas to protect against arbitrary single-qubit errors. It illustrates the idea of concatenating protection against bit and phase errors, although the physical-qubit overhead is substantial.

F.6 Stabilizer Formalism

Stabilizer methods describe quantum error-correcting codes through operators whose measurements reveal error syndromes. The encoded logical information is protected while syndrome information is obtained without directly revealing the logical state.

F.7 Surface Code

The surface code arranges data and ancilla qubits in a two-dimensional layout. Repeated stabilizer measurements detect error syndromes, and classical processing interprets these syndromes to infer likely physical errors. The approach is important because it is compatible with local interactions and is designed around fault-tolerant operation.

Physical vs Logical Qubits

A physical qubit is an individual hardware qubit. A logical qubit is encoded across multiple physical qubits so that errors can be detected and corrected. Fault-tolerant quantum computing requires additional physical resources to maintain logical reliability, creating an important overhead problem.

Part G: Integrated Lab Walkthrough

G.1 End-to-End Workflow

16. Write: construct the quantum circuit in an SDK.
17. Transpile: convert the circuit for the selected target.
18. Simulate: test ideal behavior and, where useful, a noise model.
19. Run on hardware: submit the circuit to a cloud QPU.
20. Mitigate: apply appropriate measurement/noise-mitigation methods.

21. Interpret: compare distributions and explain differences between ideal, simulated-noisy and hardware results.

G.2 Suggested Lab Exercise

Build a two-qubit Bell-state circuit. First obtain its ideal simulator histogram. Next introduce a simple noise model and observe the change. Finally, execute the circuit on a real backend if access is available. Record the backend, circuit depth, number of shots, dominant outcomes and differences from the ideal distribution.

G.3 Common Beginner Mistakes

- Confusing a statevector simulation with an experimental measurement distribution.
- Assuming every gate can be directly executed on every QPU.
- Ignoring device connectivity and the extra gates created by routing.
- Using too few shots to interpret a noisy probability distribution.
- Treating hardware results as deterministic rather than statistical.
- Comparing simulator and hardware results without checking backend noise and calibration information.

Worked Examples

Worked Example 1: Bell-State Measurement

Start with $|00\rangle$. Apply H to qubit 0 and CNOT(0,1). The final state is $|\Phi+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$.

Therefore $P(00) = |1/\sqrt{2}|^2 = 1/2$ and $P(11) = |1/\sqrt{2}|^2 = 1/2$. With 1024 ideal shots, the expected counts are approximately 512 for 00 and 512 for 11, with small statistical variation in an actual finite-shot simulation.

Worked Example 2: Statevector Storage

For n = 20 qubits, a general statevector has $2^{20} = 1{,}048{,}576$ complex amplitudes. If each complex amplitude were stored using 16 bytes, the raw statevector storage would be approximately 16,777,216 bytes, or about 16 MiB, before additional simulator overhead.

Worked Example 3: Effect of Readout Error

Suppose an ideal circuit produces state 0 with probability 0.90 and state 1 with probability 0.10. If the readout system flips 0 to 1 with probability 0.05 and 1 to 0 with probability 0.02, then the observed probability of reporting 1 is:

$P(\text{report } 1) = P(1)P(\text{report } 1|1) + P(0)P(\text{report } 1|0)$

$= 0.10(0.98) + 0.90(0.05) = 0.098 + 0.045 = 0.143.$

Thus the measured distribution can differ from the ideal distribution even when the circuit itself is correct.

Worked Example 4: Three-Qubit Bit-Flip Code

Encode logical 0 as $|000\rangle$. If the second qubit flips, the state becomes $|010\rangle$. Parity checks can reveal that the second qubit disagrees with the other two. Applying X to the second qubit restores $|000\rangle$.

State	Interpretation
000	No error / logical 0
100	Error on qubit 1
010	Error on qubit 2
001	Error on qubit 3

Worked Example 5: Transpilation and Connectivity

Suppose a logical circuit requires a CNOT between logical qubits q0 and q2, but the target hardware only directly connects q0–q1 and q1–q2. The compiler must map the logical qubits and insert routing operations so that the interaction can be implemented using available couplings. This can increase circuit depth and introduce additional opportunities for error.

Unit Summary

- Quantum SDKs provide the software layer for constructing, compiling, simulating and executing quantum circuits.
- Qiskit, Cirq, Q#, PennyLane and Braket provide different programming and execution ecosystems, while OpenQASM provides a circuit representation.
- A general n-qubit statevector contains 2^n complex amplitudes, making exact classical simulation increasingly expensive as n grows.
- Real QPUs introduce hardware constraints such as limited connectivity, native gate sets, gate errors, readout errors, T1 and T2.
- Transpilation converts an abstract circuit into a hardware-compatible circuit and can introduce routing overhead.
- Noise and decoherence arise from imperfect control and environmental interactions and affect quantum computation.
- Noise channels include bit flip, phase flip, depolarizing and amplitude damping.
- Noise mitigation techniques include measurement-error mitigation, zero-noise extrapolation, probabilistic error cancellation and dynamical decoupling.
- Quantum error correction uses encoded logical states and syndrome information to detect and correct physical errors.
- The three-qubit repetition code, Shor code, stabilizer formalism and surface-code concept provide an introductory progression toward fault-tolerant quantum computing.
- An integrated workflow is: write → transpile → simulate → run on hardware → mitigate → interpret.

Review Questions

22. What is the role of a quantum SDK in quantum computing?
23. Compare Qiskit, Cirq, Q#, PennyLane and Braket in terms of their primary roles.
24. What is OpenQASM and why is a circuit representation useful?
25. Write the basic steps for creating and executing a two-qubit circuit in Qiskit.
26. What is the difference between a statevector simulator and a real quantum processor?
27. Why does classical simulation of a general n-qubit state scale as 2^n?
28. What are qubit connectivity and native gate sets?
29. What is transpilation and why is it necessary for real hardware?
30. Differentiate T1 and T2.
31. Explain bit-flip, phase-flip, depolarizing and amplitude-damping noise.
32. What is a density matrix and why is it useful for noisy quantum systems?
33. Explain measurement-error mitigation.
34. What is zero-noise extrapolation?
35. Why is dynamical decoupling used?
36. Why is quantum error correction different from classical error correction?
37. Explain the three-qubit bit-flip repetition code with an example.
38. What is the purpose of syndrome information?
39. What is the basic idea of the nine-qubit Shor code?

40. What is the role of stabilizers in quantum error correction?
41. Differentiate physical and logical qubits.
42. Explain the basic idea of the surface code.
43. Describe the complete workflow from writing a quantum circuit to interpreting hardware results.
44. A circuit produces different histograms on an ideal simulator and a real QPU. List possible reasons for the difference.
45. How can circuit-depth reduction improve execution on noisy quantum hardware?

Suggested Practical Activities

- Create and simulate a Bell-state circuit using Qiskit.
- Compare statevector and shot-based simulation for the same circuit.
- Add a simple noise model and compare ideal and noisy histograms.
- Inspect a target backend's qubit connectivity, native gates, T1 and T2 values.
- Transpile the same circuit for two different hardware targets and compare circuit depth.
- Implement a simple three-qubit bit-flip repetition-code demonstration.
- Document an end-to-end simulator-to-hardware experiment with results and observations.